

✓ 1. Setup and basic loading

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset (adjust the path if needed)
df = pd.read_csv('AusApparalSales4thQrt2020.csv')

# Quick look at data
print("Shape:", df.shape)
print("\nInfo:")
print(df.info())
print("\nFirst 5 rows:")
display(df.head())
```

Shape: (7560, 6)

Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 7560 entries, 0 to 7559

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Date	7560 non-null	object
1	Time	7560 non-null	object
2	State	7560 non-null	object
3	Group	7560 non-null	object
4	Unit	7560 non-null	int64
5	Sales	7560 non-null	int64

dtypes: int64(2), object(4)

memory usage: 354.5+ KB

None

First 5 rows:

	Date	Time	State	Group	Unit	Sales
0	1-Oct-2020	Morning	WA	Kids	8	20000
1	1-Oct-2020	Morning	WA	Men	8	20000
2	1-Oct-2020	Morning	WA	Women	4	10000
3	1-Oct-2020	Morning	WA	Seniors	15	37500
4	1-Oct-2020	Afternoon	WA	Kids	3	7500

✓ 1. Data wrangling

- ✓ a. Ensure that the data is clean and free from any missing or incorrect entries. --> Inspect the data manually to identify missing or incorrect information using the functions `isna()` and `notna()`.

```
# Check for missing values (NaNs) in each column
print("Missing values per column:")
print(df.isna().sum())

# Check for non-missing values (sanity check)
print("\nNon-missing values per column:")
print(df.notna().sum())

# Optional: show rows that contain any missing value
missing_rows = df[df.isna().any(axis=1)]
print("\nRows with at least one missing value (if any):")
display(missing_rows)

# Basic validity checks for incorrect data
# 1) Unit and Sales should be non-negative
invalid_numeric = df[(df['Unit'] < 0) | (df['Sales'] < 0)]
print("\nRows with negative Unit or Sales (potentially incorrect):")
display(invalid_numeric)

# 2) Check unique values in categorical columns for unexpected entries or ex
print("\nUnique values in Time:")
print(df['Time'].unique())

print("\nUnique values in State:")
print(df['State'].unique())

print("\nUnique values in Group:")
print(df['Group'].unique())
```

Missing values per column:

```
Date      0
Time      0
State     0
Group     0
Unit      0
Sales     0
dtype: int64
```

Non-missing values per column:

```
Date      7560
Time      7560
State     7560
Group     7560
Unit      7560
Sales     7560
dtype: int64
```

Rows with at least one missing value (if any):

Date	Time	State	Group	Unit	Sales
------	------	-------	-------	------	-------

Rows with negative Unit or Sales (potentially incorrect):

Date	Time	State	Group	Unit	Sales
------	------	-------	-------	------	-------

Unique values in Time:

```
[' Morning' ' Afternoon' ' Evening']
```

Unique values in State:

```
[' WA' ' NT' ' SA' ' VIC' ' QLD' ' NSW' ' TAS']
```

Unique values in Group:

```
[' Kids' ' Men' ' Women' ' Seniors']
```

b. Based on your knowledge of data analytics, include your

- ▼ recommendations for treating missing and incorrect data (dropping the null values or filling them).

```
# Example: Handling missing values and incorrect data (if they existed)
```

```
# 1) If Units or Sales are missing, typical options:
```

```
# - Option A: Drop those rows (preferred for clean aggregate analysis)
# - Option B: Fill with 0 only if business logic says "no sale" when data
```

```
# Recommended code (template, only use if you actually have nulls):
```

```
df_clean = df.copy()
```

```
# Drop rows where critical numeric fields are missing
```

```
df_clean = df_clean.dropna(subset=['Unit', 'Sales'])
```

```
# If categorical fields are missing, you might fill with 'Unknown'
```

```
# df_clean['State'] = df_clean['State'].fillna('Unknown')
```

```
# df_clean['Group'] = df_clean['Group'].fillna('Unknown')
```

```
# df_clean['Time'] = df_clean['Time'].fillna('Unknown')
```

```
# 2) If negative Units or Sales are found, either:
#     - Correct from source system, or
#     - Drop as invalid records
df_clean = df_clean[(df_clean['Unit'] >= 0) & (df_clean['Sales'] >= 0)]

print("Shape after cleaning (template):", df_clean.shape)
```

Shape after cleaning (template): (7560, 6)

c. Choose a suitable data wrangling technique—either data standardization or normalization. Execute the preferred normalization method and present the resulting data. (Normalization is the preferred approach for this problem.)

```
# Work on a cleaned copy
df_norm = df_clean.copy()

# Strip leading/trailing spaces from categorical columns for consistency
for col in ['Time', 'State', 'Group']:
    df_norm[col] = df_norm[col].astype(str).str.strip()

# Convert Date to datetime for later analysis
df_norm['Date'] = pd.to_datetime(df_norm['Date'], format='%d-%b-%Y')

# Min-max normalization for Unit and Sales
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df_norm[['Unit_norm', 'Sales_norm']] = scaler.fit_transform(df_norm[['Unit',
                                                                    'Sales']])

print("Preview of normalized data:")
display(df_norm.head())
```

Preview of normalized data:

	Date	Time	State	Group	Unit	Sales	Unit_norm	Sales_norm
0	2020-10-01	Morning	WA	Kids	8	20000	0.095238	0.095238
1	2020-10-01	Morning	WA	Men	8	20000	0.095238	0.095238
2	2020-10-01	Morning	WA	Women	4	10000	0.031746	0.031746
3	2020-10-01	Morning	WA	Seniors	15	37500	0.206349	0.206349
4	2020-10-01	Afternoon	WA	Kids	3	7500	0.015873	0.015873

```
# If you prefer not to use sklearn, you can do it manually:
df_norm = df_clean.copy()
```

```

for col in ['Time', 'State', 'Group']:
    df_norm[col] = df_norm[col].astype(str).str.strip()

df_norm['Date'] = pd.to_datetime(df_norm['Date'], format='%d-%b-%Y')

# Manual min-max normalization
df_norm['Unit_norm'] = (df_norm['Unit'] - df_norm['Unit'].min()) / (df_norm['Unit'].max() - df_norm['Unit'].min())
df_norm['Sales_norm'] = (df_norm['Sales'] - df_norm['Sales'].min()) / (df_norm['Sales'].max() - df_norm['Sales'].min())

print("Preview of normalized data (manual):")
display(df_norm.head())

```

Preview of normalized data (manual):

	Date	Time	State	Group	Unit	Sales	Unit_norm	Sales_norm
0	2020-10-01	Morning	WA	Kids	8	20000	0.095238	0.095238
1	2020-10-01	Morning	WA	Men	8	20000	0.095238	0.095238
2	2020-10-01	Morning	WA	Women	4	10000	0.031746	0.031746
3	2020-10-01	Morning	WA	Seniors	15	37500	0.206349	0.206349
4	2020-10-01	Afternoon	WA	Kids	3	7500	0.015873	0.015873

d. Share your insights regarding the application of the GroupBy() function

- ✓ for either data chunking or merging, and offer a recommendation based on your analysis.

```

# Example 1: State-level aggregated sales (data chunking)
state_sales = (
    df_norm
    .groupby('State', as_index=False)
    .agg(
        total_sales=('Sales', 'sum'),
        total_units=('Unit', 'sum'),
        avg_sales=('Sales', 'mean')
    )
    .sort_values('total_sales', ascending=False)
)
print("State-level aggregated sales:")
display(state_sales)

# Example 2: State-Group level aggregation
state_group_sales = (
    df_norm
    .groupby(['State', 'Group'], as_index=False)
    .agg(
        total_sales=('Sales', 'sum'),
        total_units=('Unit', 'sum')
    )
    .sort_values(['State', 'total_sales'], ascending=[True, False])
)

```

```
)  
print("State & Group-level aggregated sales:")  
display(state_group_sales)  
  
# Example 3: Time-of-day analysis by State  
state_time_sales = (  
    df_norm  
    .groupby(['State', 'Time'], as_index=False)  
    .agg(  
        total_sales=('Sales', 'sum'),  
        total_units=('Unit', 'sum')  
    )  
)  
print("State & Time-of-day aggregated sales:")  
display(state_time_sales)
```


State-level aggregated sales:

	State	total_sales	total_units	avg_sales
5	VIC	105565000	42226	97745.370370
0	NSW	74970000	29988	69416.666667
3	SA	58857500	23543	54497.685185
2	QLD	33417500	13367	30942.129630
4	TAS	22760000	9104	21074.074074
1	NT	22580000	9032	20907.407407
6	WA	22152500	8861	20511.574074

State & Group-level aggregated sales:

	State	Group	total_sales	total_units
3	NSW	Women	19172500	7669
1	NSW	Men	19022500	7609
0	NSW	Kids	18587500	7435
2	NSW	Seniors	18187500	7275
5	NT	Men	5762500	2305
4	NT	Kids	5700000	2280
7	NT	Women	5652500	2261
6	NT	Seniors	5465000	2186
8	QLD	Kids	8510000	3404
9	QLD	Men	8392500	3357
11	QLD	Women	8325000	3330
10	QLD	Seniors	8190000	3276
15	SA	Women	14970000	5988
14	SA	Seniors	14717500	5887
13	SA	Men	14655000	5862
12	SA	Kids	14515000	5806
16	TAS	Kids	5775000	2310
17	TAS	Men	5757500	2303
18	TAS	Seniors	5650000	2260
19	TAS	Women	5577500	2231

2. Data analysis

```
import pandas as pd
import numpy as np
from scipy import stats

# Load the dataset
```

```
df = pd.read_csv('AusApparalSales4thQrt2020.csv')

# Basic cleaning (same as wrangling step)
for col in ['Time', 'State', 'Group']:
    df[col] = df[col].astype(str).str.strip()

df['Date'] = pd.to_datetime(df['Date'], format='%d-%b-%Y')

print("Data ready for analysis. Shape:", df.shape)
```

```
Data ready for analysis. Shape: (7560, 6)
```

0	NSW	Afternoon	25110000	10044
1	NSW	Evening	24712500	9885
2	NSW	Morning	25147500	10059
3	NT	Afternoon	7510000	3004
4	NT	Evening	7627500	3051

a. Perform descriptive statistical analysis on the data in the Sales and Unit columns. Utilize techniques such as mean, median, mode, and standard deviation for this analysis.

```
# Descriptive statistics using pandas
print("Descriptive statistics for Unit and Sales:")
desc_stats = df[['Unit', 'Sales']].describe()
display(desc_stats)

# Mean, median, mode, standard deviation using NumPy / SciPy
unit_mean = np.mean(df['Unit'])
unit_median = np.median(df['Unit'])
unit_mode = stats.mode(df['Unit'], keepdims=True).mode[0]
unit_std = np.std(df['Unit'], ddof=1) # sample std

sales_mean = np.mean(df['Sales'])
sales_median = np.median(df['Sales'])
sales_mode = stats.mode(df['Sales'], keepdims=True).mode[0]
sales_std = np.std(df['Sales'], ddof=1) # sample std

print("\nUnit statistics:")
print(f"Mean: {unit_mean:.2f}")
print(f"Median: {unit_median:.2f}")
print(f"Mode: {unit_mode}")
print(f"Standard Deviation: {unit_std:.2f}")

print("\nSales statistics:")
print(f"Mean: {sales_mean:.2f}")
print(f"Median: {sales_median:.2f}")
print(f"Mode: {sales_mode}")
print(f"Standard Deviation: {sales_std:.2f}")
```

Descriptive statistics for Unit and Sales:

	Unit	Sales
count	7560.000000	7560.000000
mean	18.005423	45013.558201
std	12.901403	32253.506944
min	2.000000	5000.000000
25%	8.000000	20000.000000
50%	14.000000	35000.000000
75%	26.000000	65000.000000
max	65.000000	162500.000000

Unit statistics:

Mean: 18.01

Median: 14.00

Mode: 9

Standard Deviation: 12.90

Sales statistics:

Mean: 45013.56

Median: 35000.00

Mode: 22500

Standard Deviation: 32253.51

✓ b. Identify the group with the highest sales and the group with the lowest sales based on the data provided.

```
# Aggregate total sales by customer group
group_sales = (
    df.groupby('Group', as_index=False)
      .agg(total_sales=('Sales', 'sum'),
          total_units=('Unit', 'sum'))
      .sort_values('total_sales', ascending=False)
)

print("Total sales and units by Group:")
display(group_sales)

# Identify highest and lowest sales groups
highest_group = group_sales.iloc[0]
lowest_group = group_sales.iloc[-1]

print(f"\nGroup with HIGHEST total sales: {highest_group['Group']} | Total S
print(f"Group with LOWEST total sales: {lowest_group['Group']} | Total Sales
```

Total sales and units by Group:

	Group	total_sales	total_units
1	Men	85750000	34300
3	Women	85442500	34177
0	Kids	85072500	34029
2	Seniors	84037500	33615

Group with HIGHEST total sales: Men | Total Sales: 85750000

Group with LOWEST total sales: Seniors | Total Sales: 84037500

c. Identify the group with the highest and lowest sales based on the data provided.

```
# Average sales per transaction for each group
group_avg_sales = (
    df.groupby('Group', as_index=False)
      .agg(avg_sales=('Sales', 'mean'))
      .sort_values('avg_sales', ascending=False)
)

print("Average sales per record by Group:")
display(group_avg_sales)

highest_avg_group = group_avg_sales.iloc[0]
lowest_avg_group = group_avg_sales.iloc[-1]

print(f"\nGroup with HIGHEST average sales per transaction: {highest_avg_group['Group']}")
print(f"Group with LOWEST average sales per transaction: {lowest_avg_group['Group']}")
```

Average sales per record by Group:

	Group	avg_sales
1	Men	45370.370370
3	Women	45207.671958
0	Kids	45011.904762
2	Seniors	44464.285714

Group with HIGHEST average sales per transaction: Men | Avg Sales: 45370.37

Group with LOWEST average sales per transaction: Seniors | Avg Sales: 44464.285714

d. Generate weekly, monthly, and quarterly reports to document and

present the results of the analysis conducted. (Use suitable libraries such as NumPy, Pandas, and SciPy for performing the analysis.)

```
# Ensure Date is datetime (already done above, but safe to repeat)
df['Date'] = pd.to_datetime(df['Date'])

# Week number (ISO week)
df['Week'] = df['Date'].dt.isocalendar().week

# Month name
df['Month'] = df['Date'].dt.month_name()

# Quarter period (e.g., 2020Q4)
df['Quarter'] = df['Date'].dt.to_period('Q')
```

✓ Daily report (optional but useful)

```
daily_report = (
    df.groupby('Date', as_index=False)
      .agg(
        total_sales=('Sales', 'sum'),
        total_units=('Unit', 'sum')
      )
      .sort_values('Date')
)

print("Daily sales report (head):")
display(daily_report.head())
```

Daily sales report (head):

	Date	total_sales	total_units
0	2020-10-01	3720000	1488
1	2020-10-02	3715000	1486
2	2020-10-03	3890000	1556
3	2020-10-04	3720000	1488
4	2020-10-05	3862500	1545

```
# Weekly Report:
weekly_report = (
    df.groupby('Week', as_index=False)
      .agg(
        total_sales=('Sales', 'sum'),
        total_units=('Unit', 'sum')
      )
      .sort_values('Week')
)

print("Weekly sales report:")
```

```
display(weekly_report.head())
```

Weekly sales report:

	Week	total_sales	total_units
0	40	15045000	6018
1	41	27002500	10801
2	42	26640000	10656
3	43	26815000	10726
4	44	21807500	8723

```
# Monthly report
monthly_report = (
    df.groupby('Month', as_index=False)
      .agg(
        total_sales=('Sales', 'sum'),
        total_units=('Unit', 'sum')
      )
)

# Ensure months are ordered properly (Oct, Nov, Dec)
month_order = ['October', 'November', 'December']
monthly_report['Month'] = pd.Categorical(monthly_report['Month'],
                                         categories=month_order,
                                         ordered=True)

monthly_report = monthly_report.sort_values('Month')

print("Monthly sales report:")
display(monthly_report)
```

Monthly sales report:

	Month	total_sales	total_units
2	October	114290000	45716
1	November	90682500	36273
0	December	135330000	54132

✓ 3. Data visualization

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

sns.set(style="whitegrid", palette="Set2")
```

```
# Load and clean
df = pd.read_csv('AusApparalSales4thQrt2020.csv')

for col in ['Time', 'State', 'Group']:
    df[col] = df[col].astype(str).str.strip()

df['Date'] = pd.to_datetime(df['Date'], format='%d-%b-%Y')

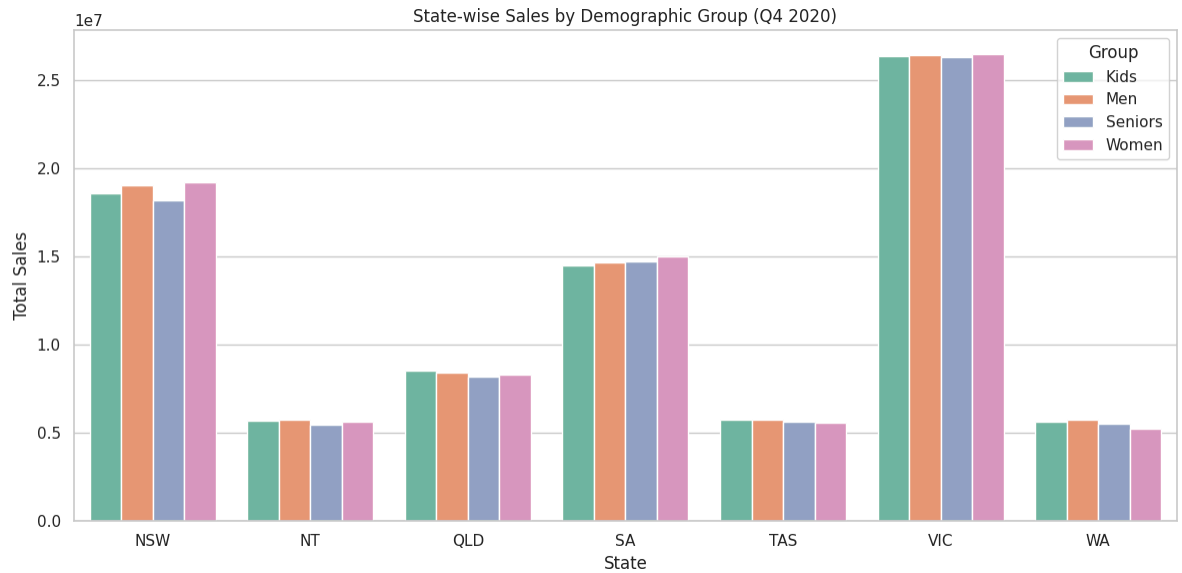
# Time features
df['Week'] = df['Date'].dt.isocalendar().week
df['Month'] = df['Date'].dt.month_name()
df['Quarter'] = df['Date'].dt.to_period('Q')
```

✓ 1. State-wise sales for each demographic group

✓ 1.1 Aggregated bar chart: State × Group

```
state_group_sales = (
    df.groupby(['State', 'Group'], as_index=False)
      .agg(total_sales=('Sales', 'sum'))
)

plt.figure(figsize=(12, 6))
sns.barplot(
    data=state_group_sales,
    x='State',
    y='total_sales',
    hue='Group'
)
plt.title('State-wise Sales by Demographic Group (Q4 2020)')
plt.xlabel('State')
plt.ylabel('Total Sales')
plt.legend(title='Group')
plt.tight_layout()
plt.show()
```



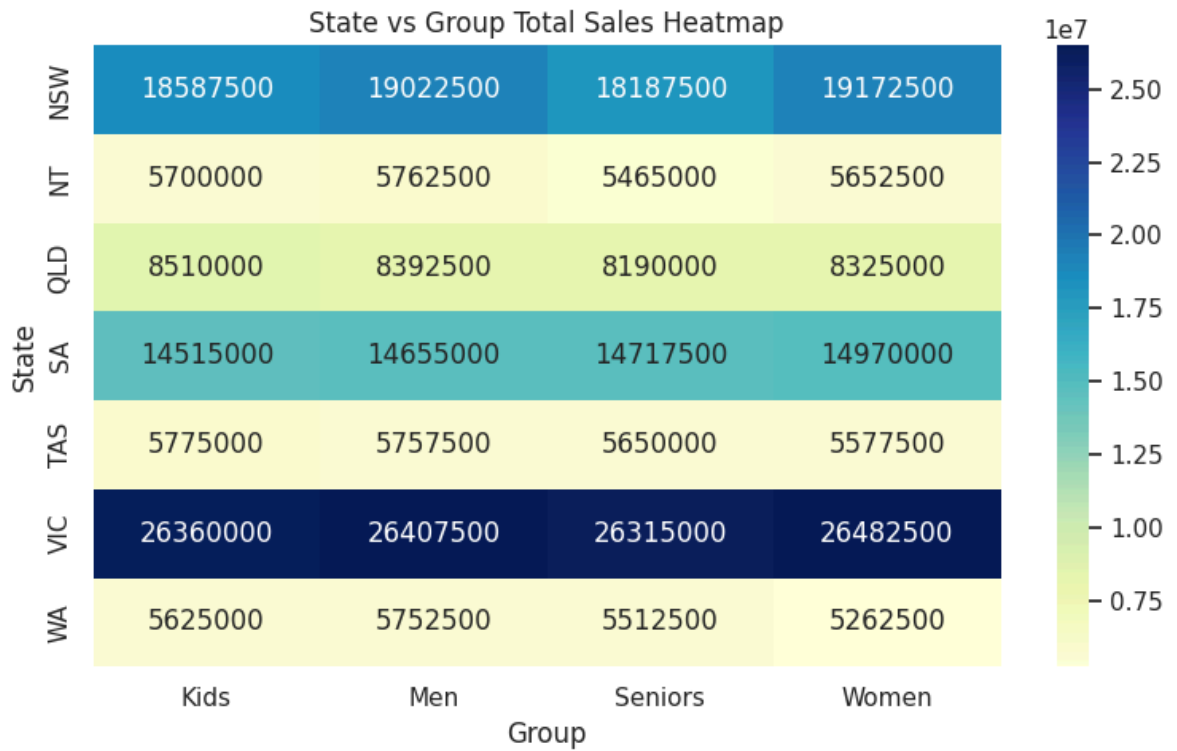
✓ 1.2 Heatmap: quick at-a-glance view

```

pivot_state_group = state_group_sales.pivot(index='State', columns='Group',

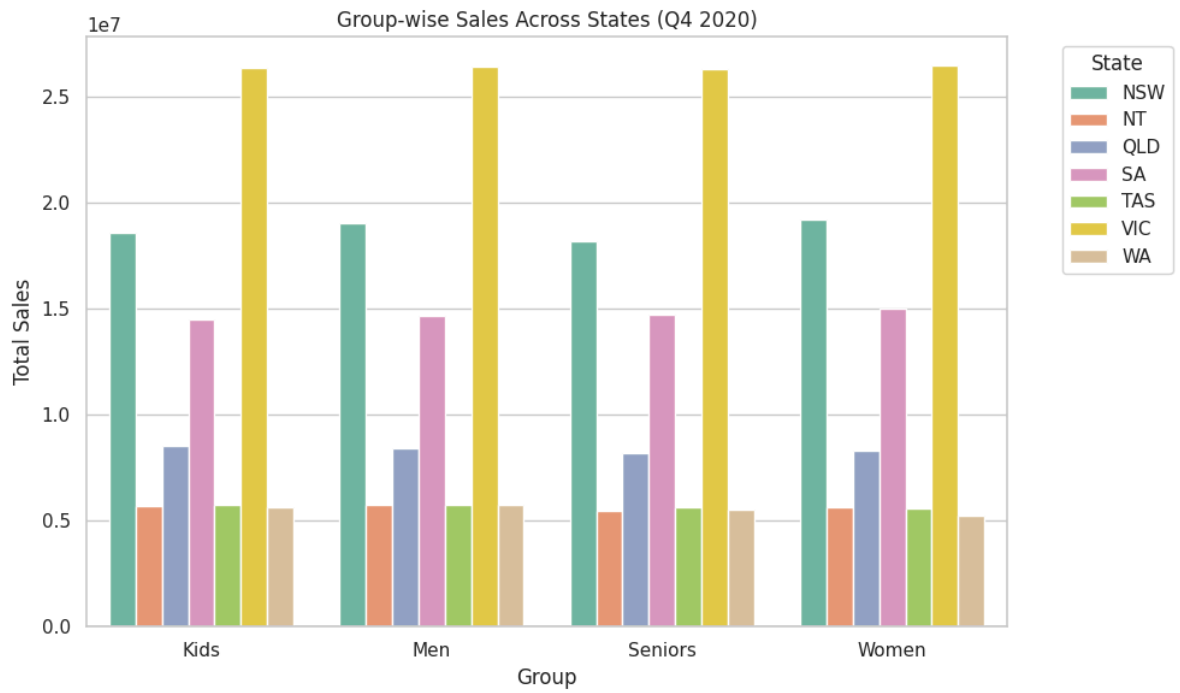
plt.figure(figsize=(8, 5))
sns.heatmap(
    pivot_state_group,
    annot=True,
    fmt='.0f',
    cmap='YlGnBu'
)
plt.title('State vs Group Total Sales Heatmap')
plt.xlabel('Group')
plt.ylabel('State')
plt.tight_layout()
plt.show()

```



2. Group-wise sales across states

```
plt.figure(figsize=(10, 6))
sns.barplot(
    data=state_group_sales,
    x='Group',
    y='total_sales',
    hue='State'
)
plt.title('Group-wise Sales Across States (Q4 2020)')
plt.xlabel('Group')
plt.ylabel('Total Sales')
plt.legend(title='State', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



OR If we want a per-group comparison with separate panels:

```
g = sns.catplot(
    data=state_group_sales,
    x='State',
    y='total_sales',
    col='Group',
    kind='bar',
    col_wrap=2,
    height=4,
    sharey=False
)
g.set_titles('Group: {col_name}')
g.set_axis_labels('State', 'Total Sales')
plt.subplots_adjust(top=0.85)
g.fig.suptitle('State-wise Sales per Group')
plt.show()
```



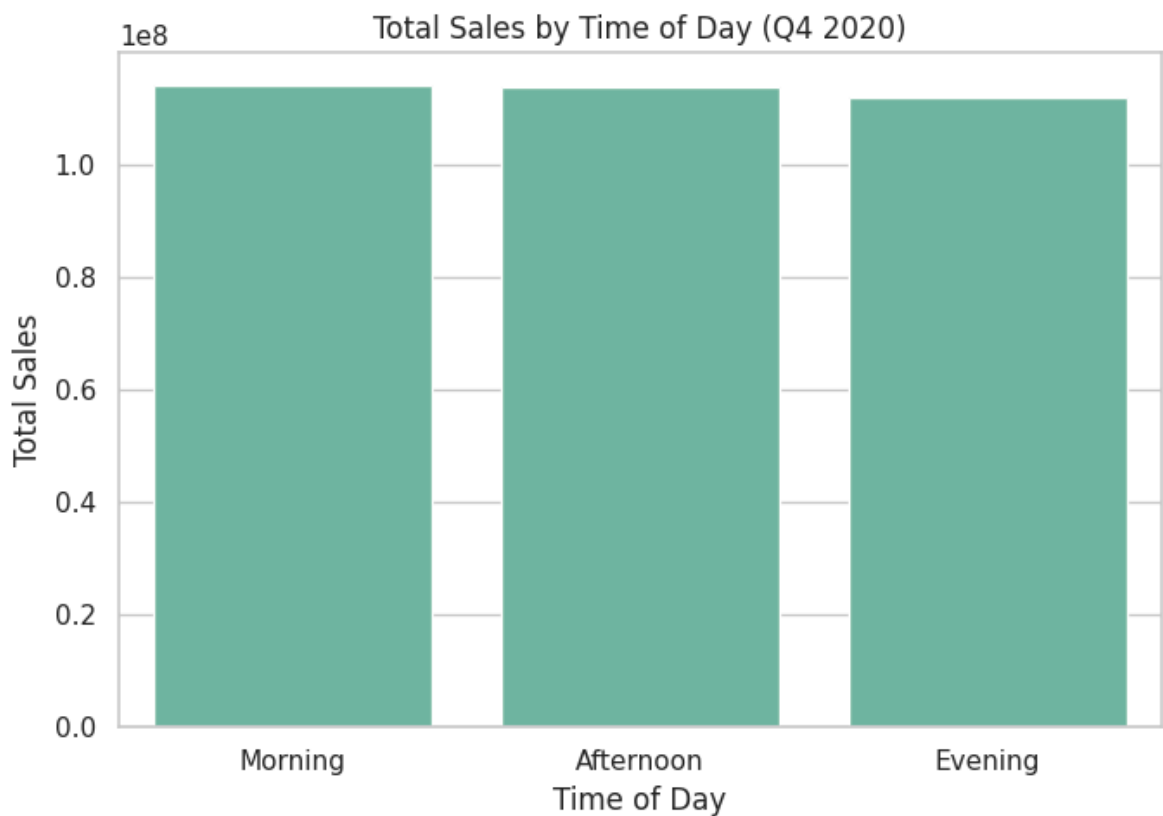
✓ 3. Time-of-day analysis (peak and off-peak)

✓ 3.1 Overall time-of-day performance

```
time_sales = (
    df.groupby('Time', as_index=False)
      .agg(total_sales=('Sales', 'sum'))
      .sort_values('total_sales', ascending=False)
)
```

```
plt.figure(figsize=(7, 5))
sns.barplot(
    data=time_sales,
    x='Time',
    y='total_sales',
    order=['Morning', 'Afternoon', 'Evening'] # adjust if needed
)
plt.title('Total Sales by Time of Day (Q4 2020)')
plt.xlabel('Time of Day')
plt.ylabel('Total Sales')
plt.tight_layout()
plt.show()

print("Time-of-day sales:")
display(time_sales)
```

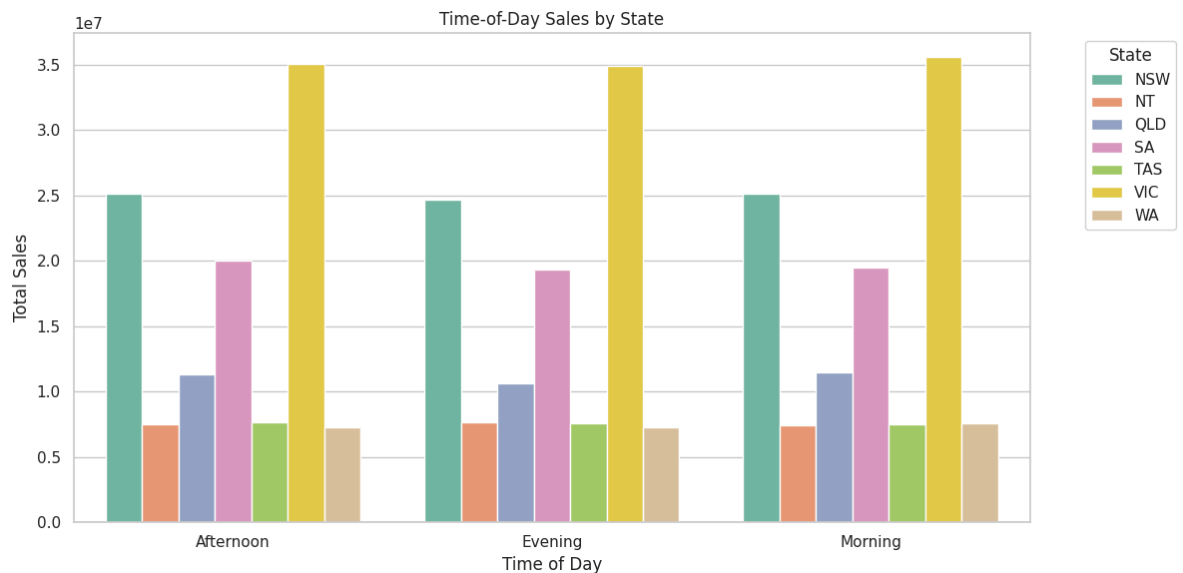


Time-of-day sales:

	Time	total_sales
2	Morning	114207500
0	Afternoon	114007500
1	Evening	112087500

3.2 Time-of-day by State (for targeted programs)

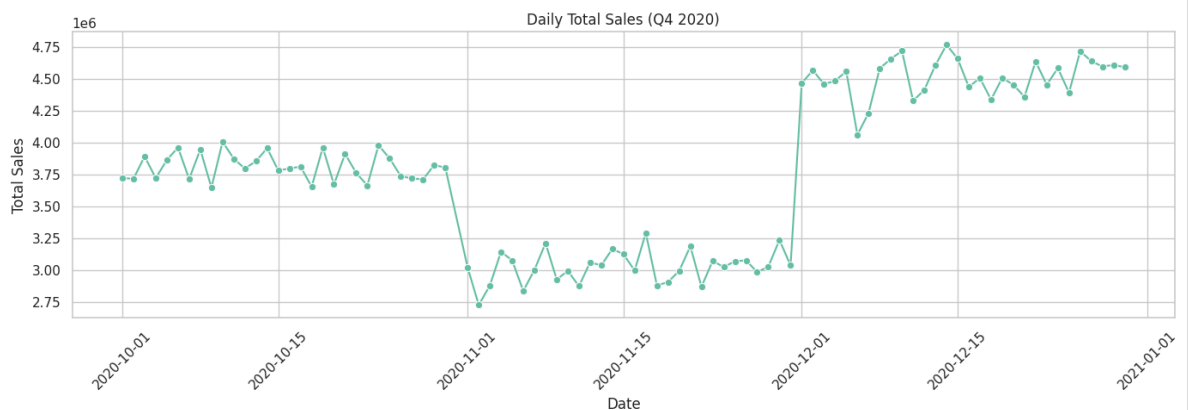
```
time_state_sales = (  
    df.groupby(['Time', 'State'], as_index=False)  
      .agg(total_sales=('Sales', 'sum'))  
)  
  
plt.figure(figsize=(12, 6))  
sns.barplot(  
    data=time_state_sales,  
    x='Time',  
    y='total_sales',  
    hue='State'  
)  
plt.title('Time-of-Day Sales by State')  
plt.xlabel('Time of Day')  
plt.ylabel('Total Sales')  
plt.legend(title='State', bbox_to_anchor=(1.05, 1), loc='upper left')  
plt.tight_layout()  
plt.show()
```



✓ 4. Daily, weekly, monthly, quarterly charts

4.1 Daily sales trend

```
daily_sales = (  
    df.groupby('Date', as_index=False)  
        .agg(total_sales=('Sales', 'sum'))  
        .sort_values('Date')  
)  
  
plt.figure(figsize=(14, 5))  
sns.lineplot(  
    data=daily_sales,  
    x='Date',  
    y='total_sales',  
    marker='o'  
)  
plt.title('Daily Total Sales (Q4 2020)')  
plt.xlabel('Date')  
plt.ylabel('Total Sales')  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```



4.2 Weekly sales (overall)

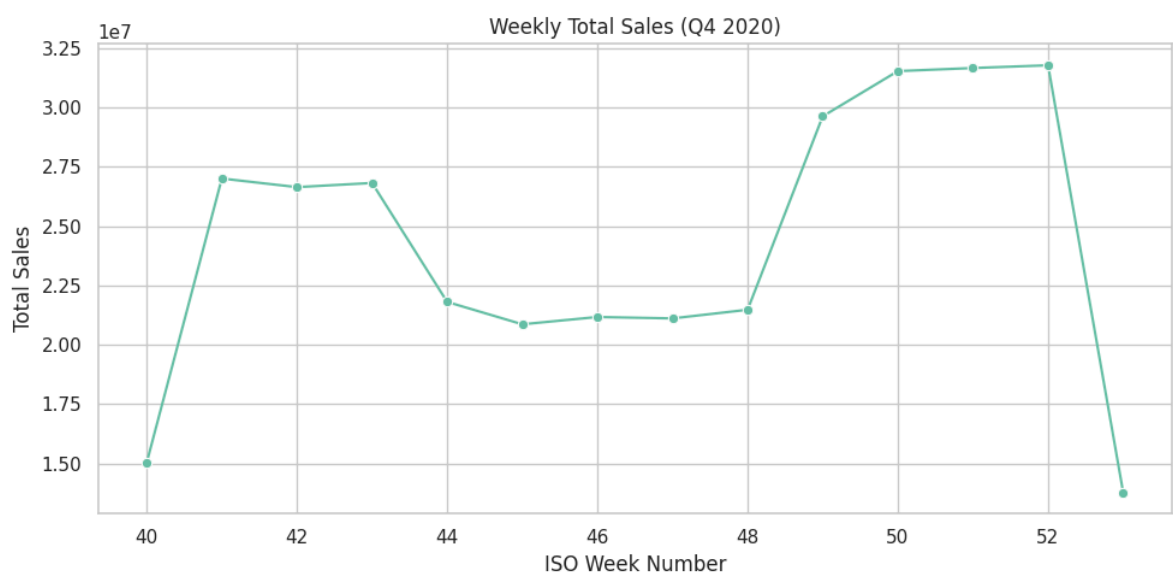
```
weekly_sales = (  
    df.groupby('Week', as_index=False)  
        .agg(total_sales=('Sales', 'sum'))  
)
```

```

        .sort_values('Week')
    )

plt.figure(figsize=(10, 5))
sns.lineplot(
    data=weekly_sales,
    x='Week',
    y='total_sales',
    marker='o'
)
plt.title('Weekly Total Sales (Q4 2020)')
plt.xlabel('ISO Week Number')
plt.ylabel('Total Sales')
plt.tight_layout()
plt.show()

```



✓ 4.3 Weekly sales by State (for S&M planning)

```

weekly_state_sales = (
    df.groupby(['Week', 'State'], as_index=False)
        .agg(total_sales=('Sales', 'sum'))
        .sort_values(['Week', 'State'])
)

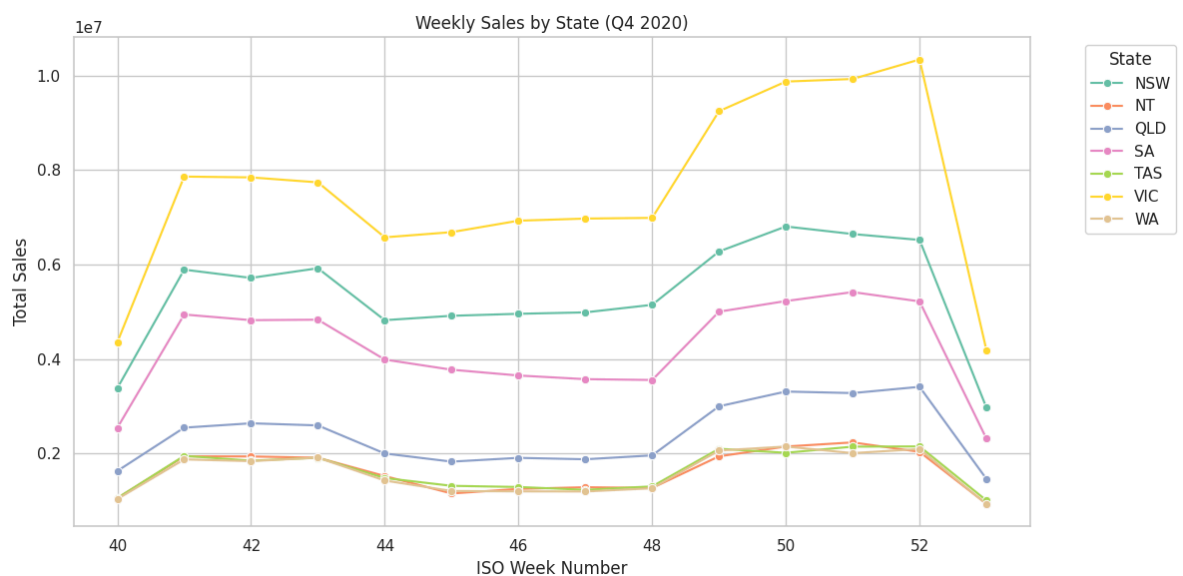
plt.figure(figsize=(12, 6))

```

```

sns.lineplot(
    data=weekly_state_sales,
    x='Week',
    y='total_sales',
    hue='State',
    marker='o'
)
plt.title('Weekly Sales by State (Q4 2020)')
plt.xlabel('ISO Week Number')
plt.ylabel('Total Sales')
plt.legend(title='State', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()

```



✓ 4.4 Monthly sales (overall)

```

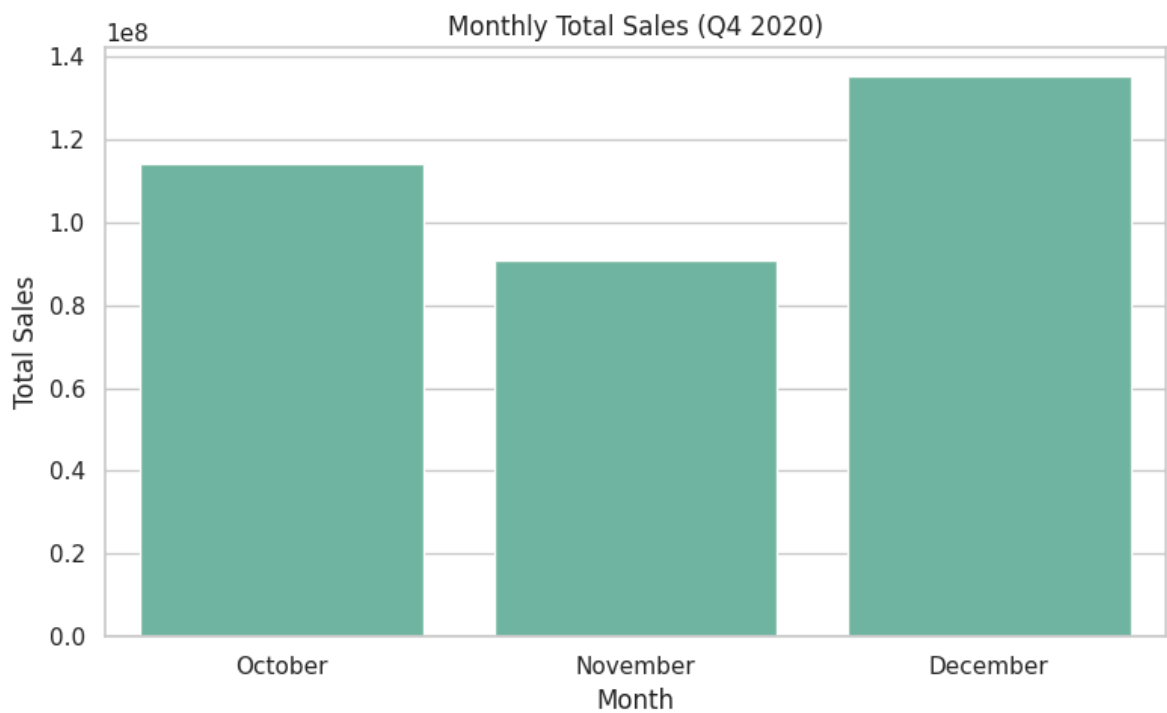
monthly_sales = (
    df.groupby('Month', as_index=False)
        .agg(total_sales=('Sales', 'sum'))
)

```

```
month_order = ['October', 'November', 'December']
monthly_sales['Month'] = pd.Categorical(monthly_sales['Month'],
                                       categories=month_order,
                                       ordered=True)

monthly_sales = monthly_sales.sort_values('Month')

plt.figure(figsize=(8, 5))
sns.barplot(
    data=monthly_sales,
    x='Month',
    y='total_sales'
)
plt.title('Monthly Total Sales (Q4 2020)')
plt.xlabel('Month')
plt.ylabel('Total Sales')
plt.tight_layout()
plt.show()
```



✓ 4.5 Monthly sales by State

```
monthly_state_sales = (
    df.groupby(['Month', 'State'], as_index=False)
        .agg(total_sales=('Sales', 'sum'))
)
```

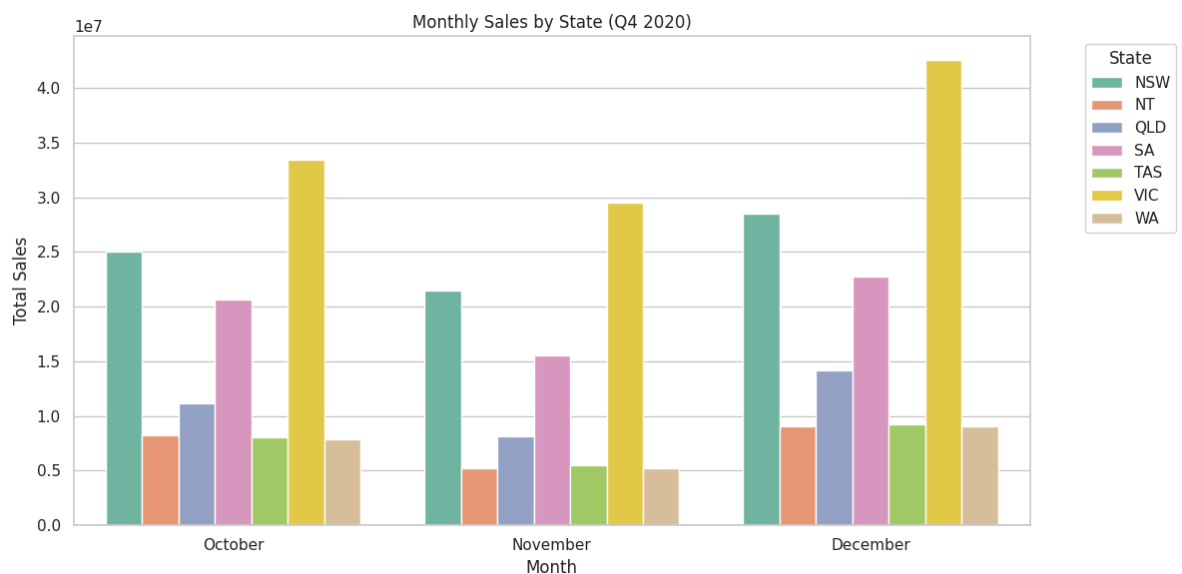
```

monthly_state_sales['Month'] = pd.Categorical(monthly_state_sales['Month'],
                                              categories=month_order,
                                              ordered=True)

monthly_state_sales = monthly_state_sales.sort_values(['Month', 'State'])

plt.figure(figsize=(12, 6))
sns.barplot(
    data=monthly_state_sales,
    x='Month',
    y='total_sales',
    hue='State'
)
plt.title('Monthly Sales by State (Q4 2020)')
plt.xlabel('Month')
plt.ylabel('Total Sales')
plt.legend(title='State', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()

```



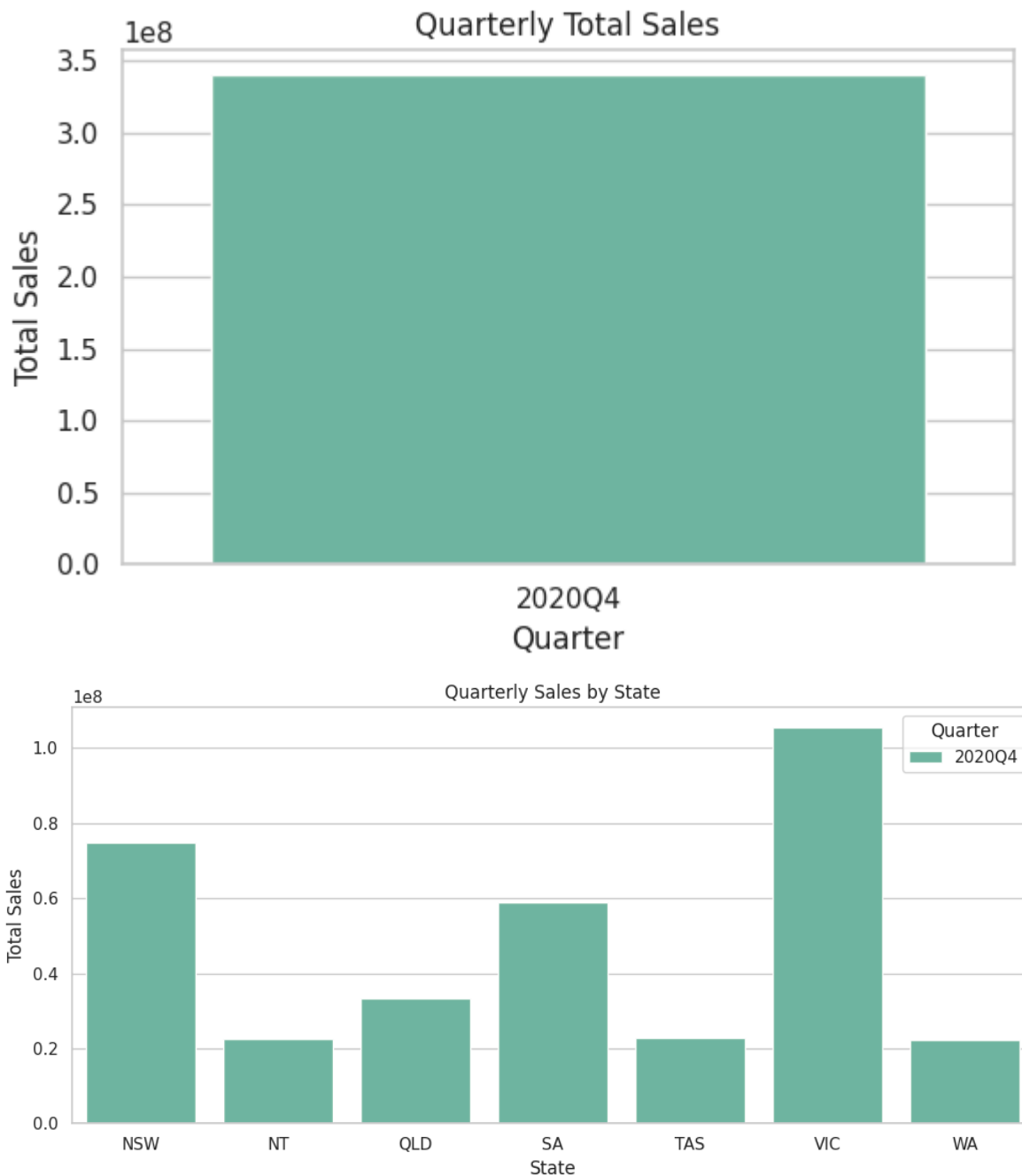
✓ 4.6 Quarterly sales (overall and by state)

```
quarterly_sales = (
    df.groupby('Quarter', as_index=False)
      .agg(total_sales=('Sales', 'sum'))
)

plt.figure(figsize=(6, 4))
sns.barplot(
    data=quarterly_sales,
    x='Quarter',
    y='total_sales'
)
plt.title('Quarterly Total Sales')
plt.xlabel('Quarter')
plt.ylabel('Total Sales')
plt.tight_layout()
plt.show()

quarterly_state_sales = (
    df.groupby(['Quarter', 'State'], as_index=False)
      .agg(total_sales=('Sales', 'sum'))
)

plt.figure(figsize=(10, 5))
sns.barplot(
    data=quarterly_state_sales,
    x='State',
    y='total_sales',
    hue='Quarter'
)
plt.title('Quarterly Sales by State')
plt.xlabel('State')
plt.ylabel('Total Sales')
plt.tight_layout()
plt.show()
```



4. Report generation

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
sns.set(style="whitegrid", palette="Set2")

df = pd.read_csv('AusApparalSales4thQrt2020.csv')

for col in ['Time', 'State', 'Group']:
    df[col] = df[col].astype(str).str.strip()

df['Date'] = pd.to_datetime(df['Date'], format='%d-%b-%Y')
df['Week'] = df['Date'].dt.isocalendar().week
df['Month'] = df['Date'].dt.month_name()
df['Quarter'] = df['Date'].dt.to_period('Q')
```

✓ 2. Box plots for descriptive statistics

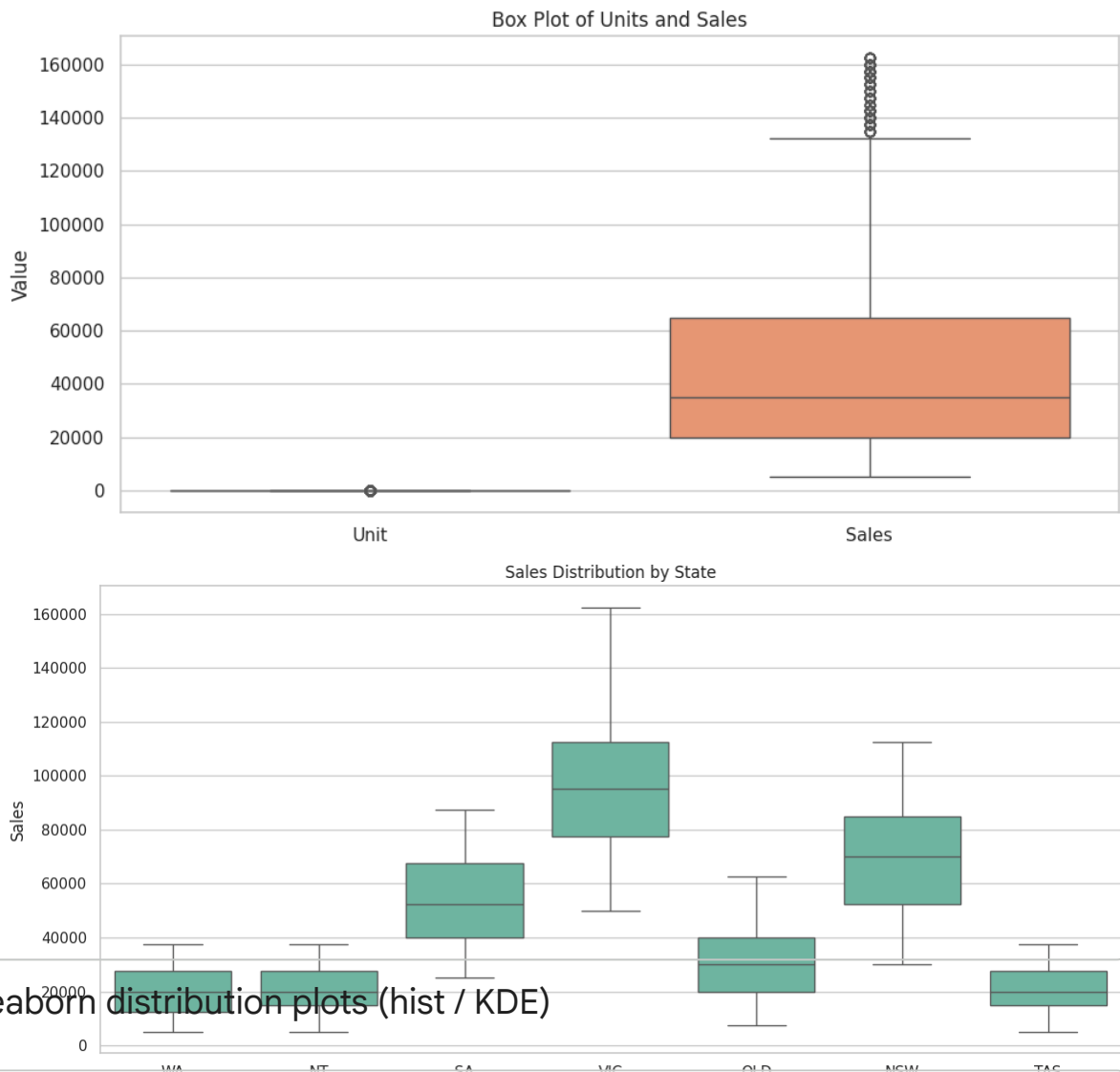
```
### Box plots for Unit and Sales

# The box plots summarize the distribution (median, spread, and outliers) fo

plt.figure(figsize=(10, 5))
sns.boxplot(data=df[['Unit', 'Sales']])
plt.title('Box Plot of Units and Sales')
plt.ylabel('Value')
plt.tight_layout()
plt.show()

# Optional: box plots by State or Group
plt.figure(figsize=(12, 6))
sns.boxplot(data=df, x='State', y='Sales')
plt.title('Sales Distribution by State')
plt.xlabel('State')
plt.ylabel('Sales')
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x='Group', y='Sales')
plt.title('Sales Distribution by Group')
plt.xlabel('Group')
plt.ylabel('Sales')
plt.tight_layout()
plt.show()
```

3. Seaborn distribution plots (hist / KDE)

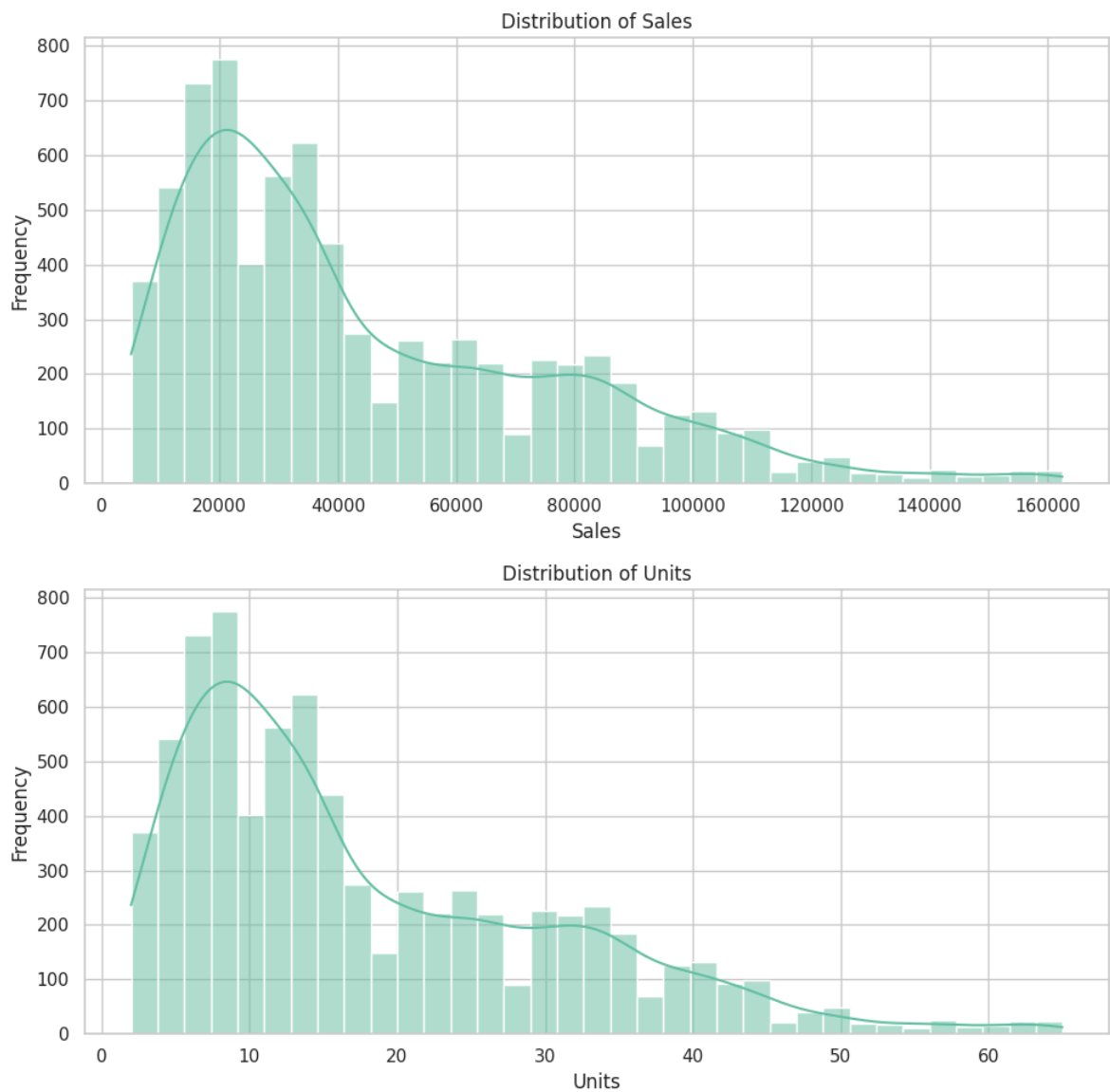
```

### Distribution of Sales and Units
# These Seaborn distribution plots show how Sales and Units are distributed,

# Sales distribution
plt.figure(figsize=(10, 5))
sns.histplot(df['Sales'], kde=True)
plt.title('Distribution of Sales')
plt.xlabel('Sales')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

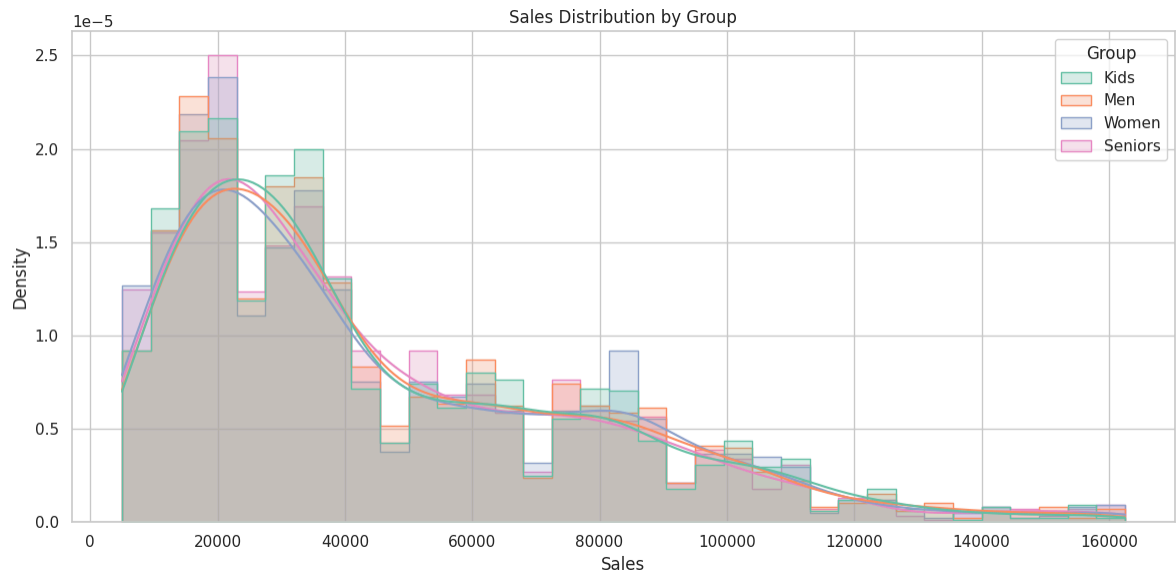
# Units distribution
plt.figure(figsize=(10, 5))
sns.histplot(df['Unit'], kde=True)
plt.title('Distribution of Units')
plt.xlabel('Units')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

```



distributions by subgroup

```
plt.figure(figsize=(12, 6))
sns.histplot(data=df, x='Sales', hue='Group', kde=True, element='step', stat
plt.title('Sales Distribution by Group')
plt.xlabel('Sales')
plt.ylabel('Density')
plt.tight_layout()
plt.show()
```



✓ 4. Integrating earlier visualizations into the report

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set(style="whitegrid", palette="Set2")

# --- 1. State-wise sales by group ---
state_group_sales = (
    df.groupby(['State', 'Group'], as_index=False)
      .agg(total_sales=('Sales', 'sum'))
)
```